
name,diameter,distance
Mercury,4879,57.9
Venus,12104,108.2
Earth,12756,149.6
Mars,6792,228.0
-->
-->
collage
IN 10 PARTS
part 4 of the course: after advanced
1 reading a CSV file
2 from text to numbers
3 fitting numbers: map
4 your copy: editing the data
5 data from the internet: curl
6 your own data
7 a folder full of images
8 random compositions
9 your own images
10 a collage tool
your programs read FILES: tables of data and folders of images.
the final project: a collage made from your own images and your own data!

PART 1 / 10 — files: data and images

# READING A CSV FILE

new: csvReader, readAllWithHeader, getValue

planets.csv:

```
+------------------------+
| name,type,diameter_km  | <- names
| Mercury,rocky,4879     | <- row 0
| Venus,rocky,12104      | <- row 1
| Earth,rocky,12756      |
+------------------------+
```

comma separated values:
a table, written as plain text

```
row.getValue("name")  ->  "Mercury"
everything is TEXT, even "4879"
```

```
* CSV: a table in a text file
* the first line has the column names
* println(...) writes to the console

val rows = csvReader()
    .readAllWithHeader(
        File("data/csv/planets.csv"))
for (row in rows) {
    drawer.text(row.getValue("name"), ...)
}
```

```
+-- EXERCISE 1 -------------------------------+
| a) look at the console                       |
| b) show mean_temp_c, with °C                 |
| c) only the gas giants                       |
| d) read image-archive.csv                    |
+----------------------------------------------+
```

```
  ___   ____        PART 2 / 10                                  files: data and images
 / _ \ |___ \
| | | |  __) |     FROM TEXT TO NUMBERS
| |_| | / __/      ~~~~~~~~~~~~~~~~~~~~
 \___/ |_____|     new: toDouble(), drawing with data
```

```
"142984"  .toDouble()  ->  142984.0
 text                       number
```

```
142984.0 / 1000.0  =  143 pixels
```

```
              .-----.
           .'       '.    .---.
  . . . .  |  Jupiter  |  ( Sat )  o  o
           '.       .'    '---'
             '-----'
```

divide everything by the same number:
the sizes stay honest

```
* text from a file -> toDouble()
* 1 pixel = 1000 km
* a var x: where the next planet goes

val diameter =
    row.getValue("diameter_km").toDouble()
val size = diameter / 1000.0
drawer.circle(x + size / 2.0, 360.0,
              size / 2.0)
x = x + size + 25.0
```

```
+-- EXERCISE 2 ------------------------------+
| a) divide by 800 or 2000                   |
| b) small to big: sortedBy { }              |
| c) the length of a day as bars             |
| d) add up all diameters                    |
+--------------------------------------------+
```

PART 3 / 10

files: data and images



# FITTING NUMBERS IN THE WINDOW

new: map(...), colours from numbers

```
map(57.9, 4515.0, 60.0, 660.0, distance)

from: 57.9 ---------------------- 4515.0
        |                          |
        v                          v
to:     60 ---------------------- 660
```

like reading a ruler

temperature -> colour:

```
BLUE.mix(RED, t)
t = 0.0 [####......] t = 1.0
  cold                 hot
```

```
* map: from one range to another
* find min and max with a loop
* mouse over a planet: its numbers

val x = map(minDistance, maxDistance,
            60.0, 660.0, distance)
val t = map(minTemp, maxTemp,
            0.0, 1.0, temp)
drawer.fill =
    ColorRGBa.BLUE.mix(ColorRGBa.RED, t)
```

```
+-- EXERCISE 3 ------------------------------+
| a) use sqrt(distance)                      |
| b) a vertical line                         |
| c) day_hours as the radius                 |
| d) orbits around the Sun                   |
+--------------------------------------------+
```

PART 4 / 10 — files: data and images

## YOUR COPY: EDITING THE DATA

new: exists(), toDoubleOrNull(), ?:

```
planets.csv  --copy-->  my-planets.csv
                        + Pluto
                        + a color column
```

```
* if my-planets.csv exists, use it
* null: no value, nothing
* a ?: b  -  'a, or else b'

val distance = row["distance_million_km"]
    ?.toDoubleOrNull()
if (distance == null) {
    problems = problems + 1
    continue      // skip this row
}
```

```
"12x04".toDouble()        crash!
"12x04".toDoubleOrNull()  null
row["color"] ?: "pink"
```

**EXERCISE 4**

a) add Pluto. does it fit?
b) a color column: #3366ff
c) a typo on purpose
d) invent your own planet

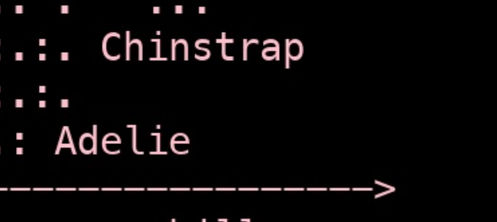
PART 5 / 10 — files: data and images

# DATA FROM THE INTERNET

new: curl, a scatter plot, missing values

```
$ curl -L -o data/csv/penguins.csv \
  https://raw.githubusercontent.com/
 mwaskom/seaborn-data/master/penguins.csv
```

(Windows: curl.exe)




* curl downloads a file
* -L follow, -o save as
* empty cells: toDoubleOrNull()

```
val bill = row.getValue("bill_length_mm")
    .toDoubleOrNull()
if (bill == null) continue
val x = map(30.0, 60.0, 80.0, 680.0, bill)
```

**EXERCISE 5**

a) body_mass_g as y
b) bigger dots for males
c) only one island
d) find your own CSV online

PART 6 / 10 — files: data and images



# YOUR OWN DATA

new: column names, ; separators, arrow keys

my-data.csv

```
day;sleep;tea;mood
mon;7,5;3;6
tue;6;5;4
```

; instead of , -> delimiter = ';'

7,5 instead of 7.5 -> replace(',', '.')

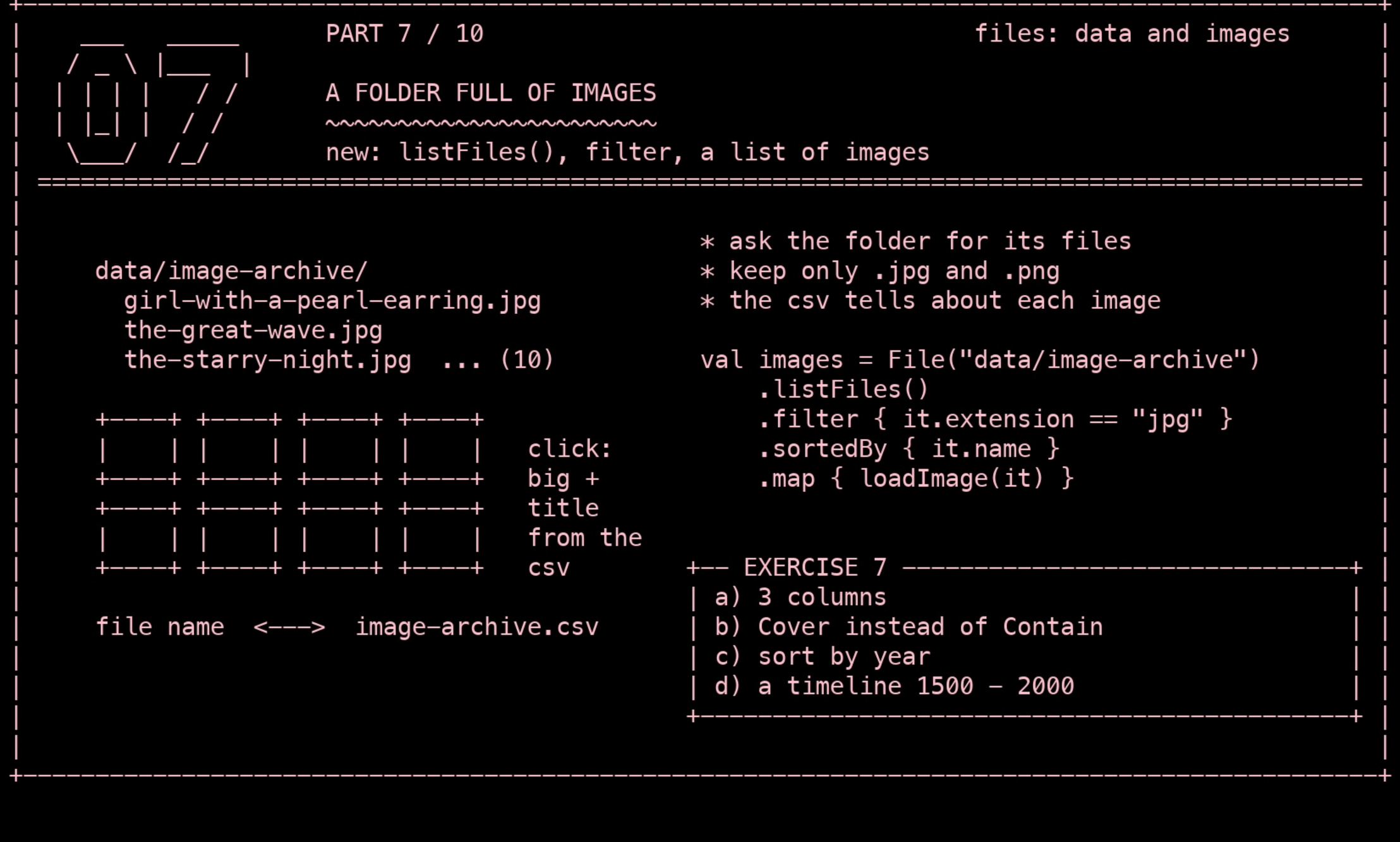
```
mon  [########...]  7.5
tue  [######.....]  6.0
          <- -> other column
```

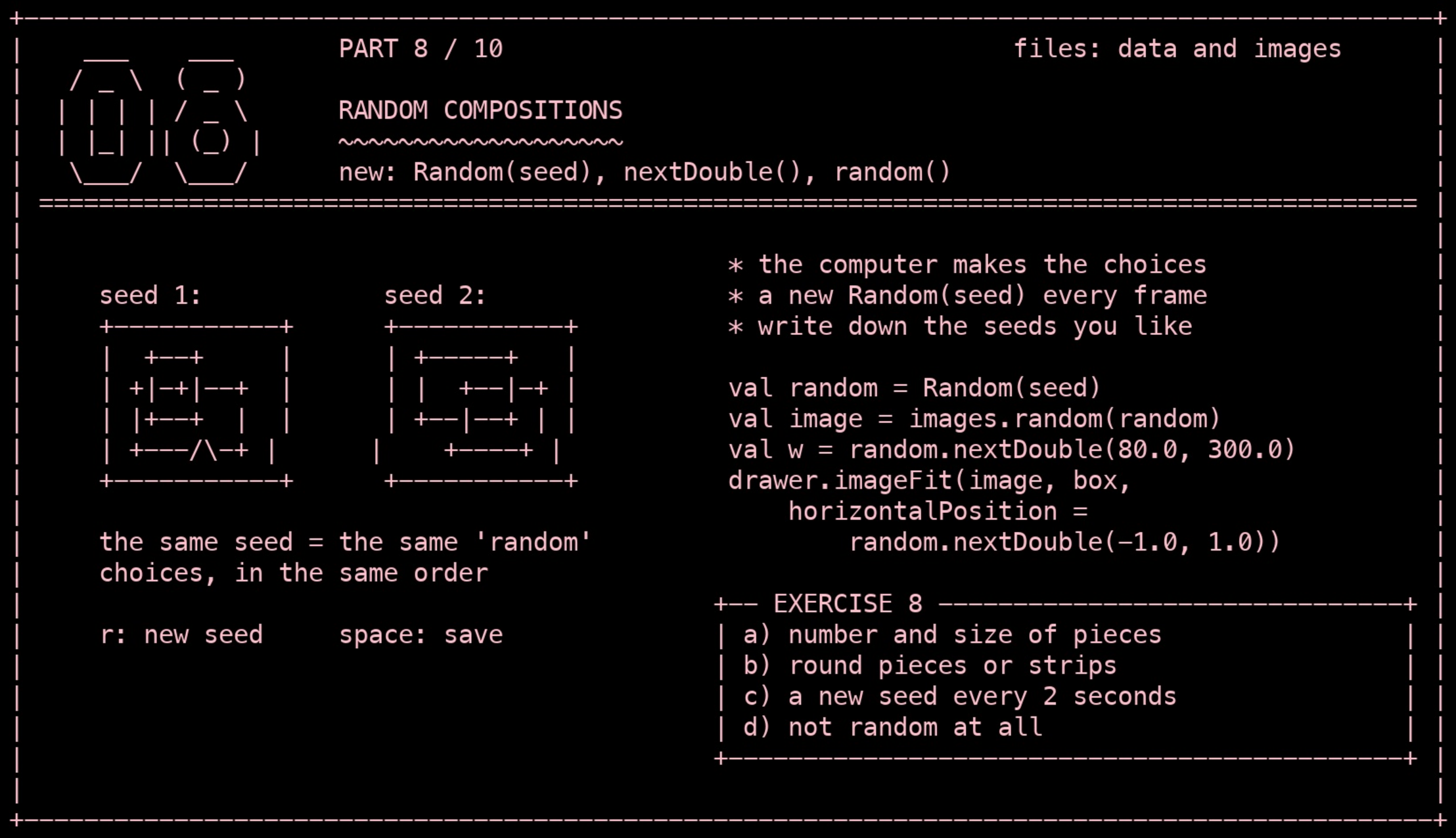
```
* rows[0].keys: the column names
* find the columns with only numbers
* spreadsheets: export as CSV

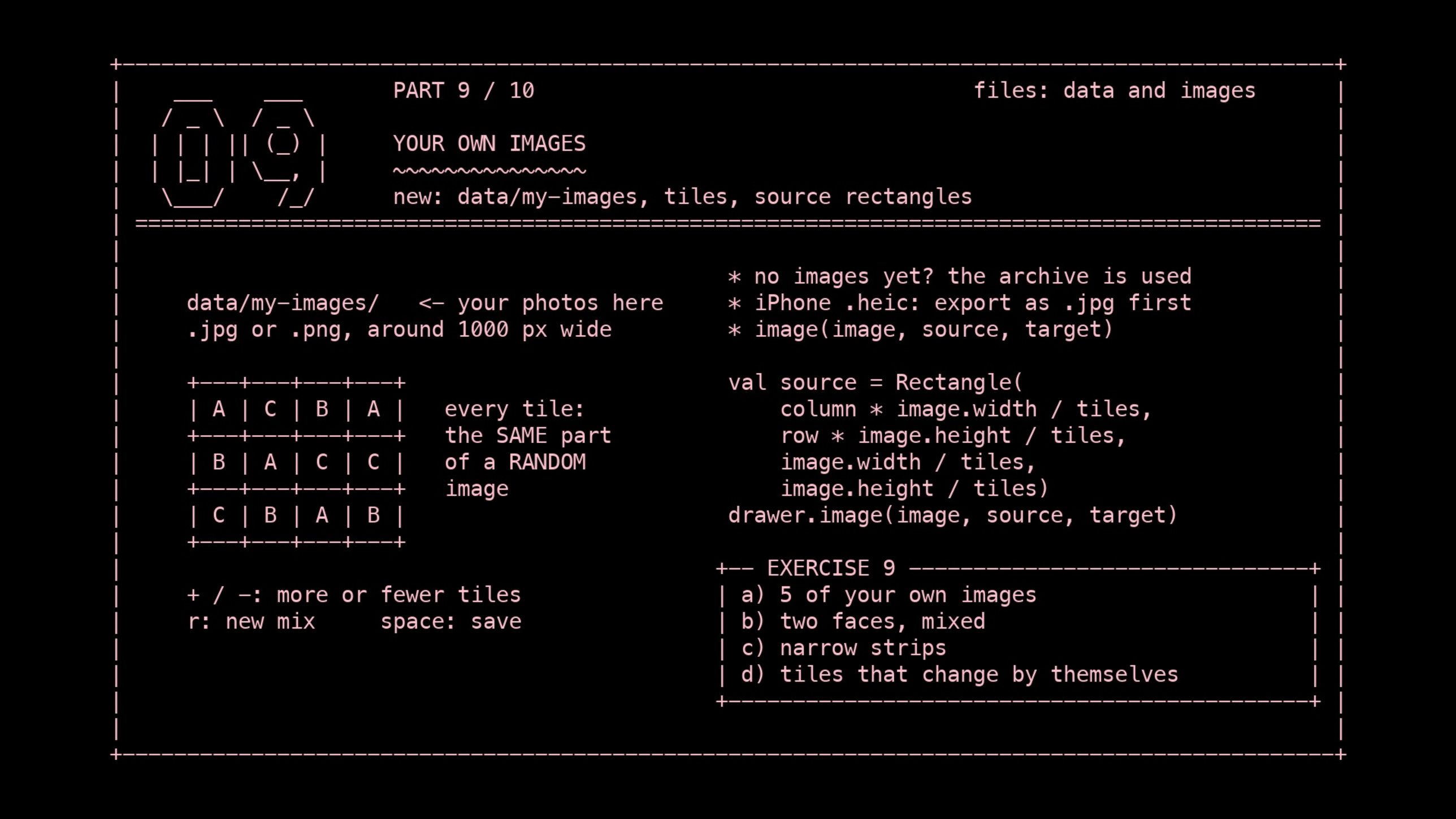
val separator =
    if (firstLine.contains(";")) ';'
    else ','
csvReader { delimiter = separator }
    .readAllWithHeader(file)
```

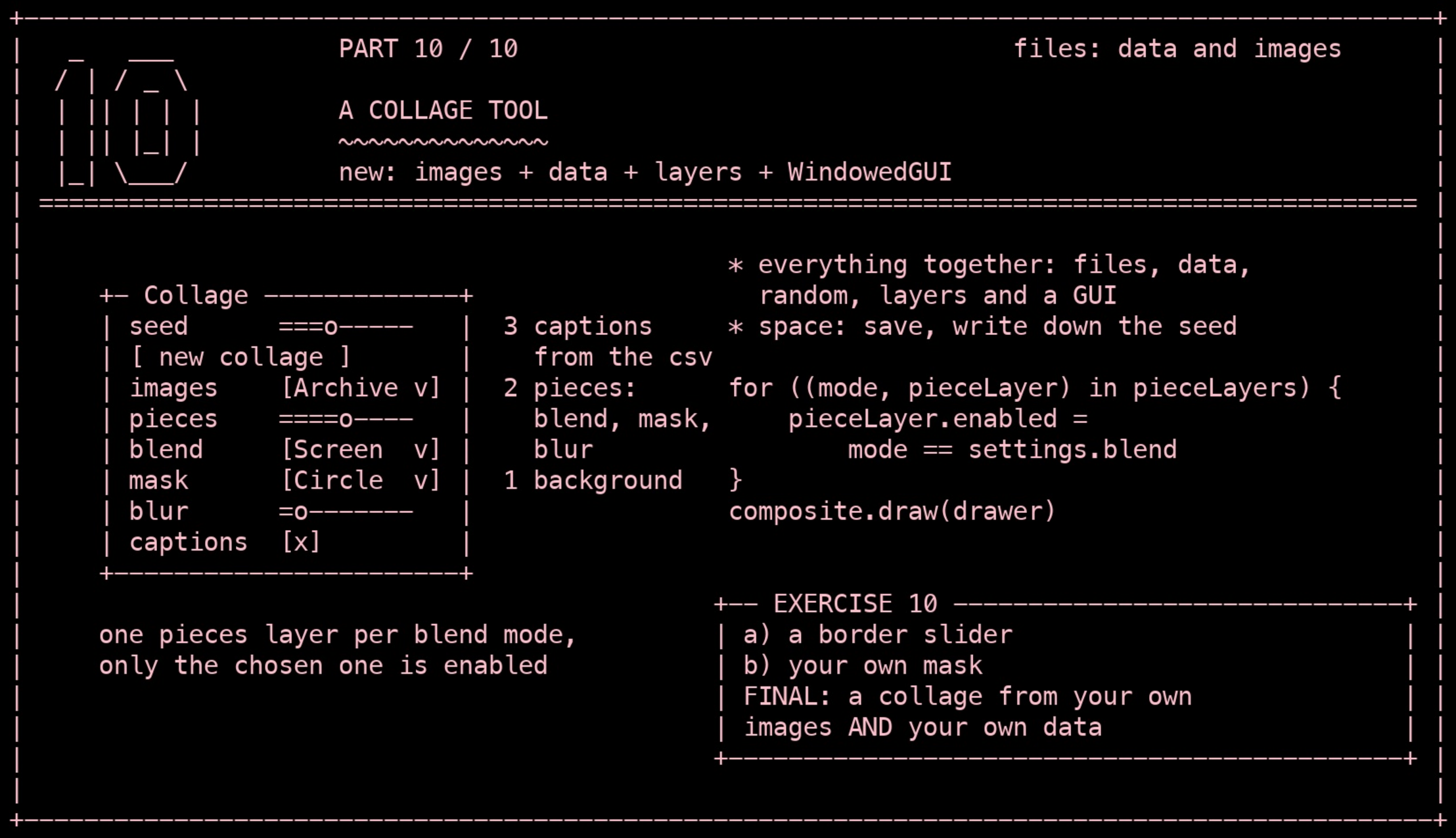
## EXERCISE 6

a) make my-data.csv
b) export from a spreadsheet
c) circles instead of bars
d) a picture that fits the data


PART 7 / 10
files: data and images
A FOLDER FULL OF IMAGES
new: listFiles(), filter, a list of images
data/image-archive/
girl-with-a-pearl-earring.jpg
the-great-wave.jpg
the-starry-night.jpg ... (10)
click:
big +
title
from the
csv
file name <---> image-archive.csv
* ask the folder for its files
* keep only .jpg and .png
* the csv tells about each image
val images = File("data/image-archive")
.listFiles()
.filter { it.extension == "jpg" }
.sortedBy { it.name }
.map { loadImage(it) }
EXERCISE 7
a) 3 columns
b) Cover instead of Contain
c) sort by year
d) a timeline 1500 - 2000


PART 8 / 10
files: data and images
RANDOM COMPOSITIONS
new: Random(seed), nextDouble(), random()
seed 1:
seed 2:
the same seed = the same 'random'
choices, in the same order
r: new seed
space: save
* the computer makes the choices
* a new Random(seed) every frame
* write down the seeds you like
val random = Random(seed)
val image = images.random(random)
val w = random.nextDouble(80.0, 300.0)
drawer.imageFit(image, box,
horizontalPosition =
random.nextDouble(-1.0, 1.0))
EXERCISE 8
a) number and size of pieces
b) round pieces or strips
c) a new seed every 2 seconds
d) not random at all


PART 9 / 10
files: data and images
YOUR OWN IMAGES
new: data/my-images, tiles, source rectangles
data/my-images/ <- your photos here
.jpg or .png, around 1000 px wide
every tile:
the SAME part
of a RANDOM
image
+ / -: more or fewer tiles
r: new mix space: save
* no images yet? the archive is used
* iPhone .heic: export as .jpg first
* image(image, source, target)
val source = Rectangle(
column * image.width / tiles,
row * image.height / tiles,
image.width / tiles,
image.height / tiles)
drawer.image(image, source, target)
EXERCISE 9
a) 5 of your own images
b) two faces, mixed
c) narrow strips
d) tiles that change by themselves


PART 10 / 10
files: data and images
A COLLAGE TOOL
new: images + data + layers + WindowedGUI
+- Collage ----------+
| seed ===o----- |
| [ new collage ] |
| images [Archive v] |
| pieces ====o---- |
| blend [Screen v] |
| mask [Circle v] |
| blur =o------ |
| captions [x] |
3 captions from the csv
2 pieces: blend, mask, blur
1 background
* everything together: files, data, random, layers and a GUI
* space: save, write down the seed
for ((mode, pieceLayer) in pieceLayers) {
pieceLayer.enabled =
mode == settings.blend
}
composite.draw(drawer)
one pieces layer per blend mode,
only the chosen one is enabled
+-- EXERCISE 10 ----------
| a) a border slider
| b) your own mask
| FINAL: a collage from your own
| images AND your own data